Implementation and Analysis of Neural Networks from Scratch and Using PyTorch on the Fashion-MNIST Dataset

Siddh Sharma

Roll No. 25135150

Mechanical Engineering

Indian Institute of Technology (BHU), Varanasi

1 Introduction

The objective of this report is to implement an Artificial Neural Network for the Fashion-MNIST dataset from scratch using NumPy and compare its performance with an equivalent PyTorch implementation.

Fashion-MNIST consists of grayscale images of clothing items belonging to ten categories including T-shirts, Trousers, Pullovers, Dresses, Coats, Sandals, Shirts, Sneakers, Bags and Ankle Boots. The dataset is considerably more challenging than MNIST because several classes share similar visual characteristics.

The report focuses on architecture selection, optimizer experimentation, implementation challenges and performance analysis. Different architectures and optimization techniques were evaluated and compared on the basis of accuracy, convergence time and memory usage before selecting the final model.

2 Architecture Selection

The first was to decide an appropriate neural network architecture.

Due to the very high convergence time of the NumPy implementation,I used Pytorch implementation to compare validation accuracies,convergence time and memory used.

I initially started with a two-hidden-layer architecture containing 50 neurons in each hidden layer. This architecture was able to achieve a validation accuracy of 84.68%, which was not a bad initial validation accuracy.

To improve the accuracy, I increased the number of neurons and decided to try on a 64-64 architecture. This architecture achieved a validation accuracy of 84.39%. This showed that there was probably overfitting as the validation accuracy decreased here.

I then increased the size of the first hidden layer and evaluated a 128-64 architecture to see if the accuracy increased. This architecture achieved the highest validation accuracy of 85.43

A larger architecture such as 128-128 was not used as it took a lot of computation time with very little or no increase in validation accuracy. Since the 128-64 architecture achieved the highest validation accuracy among all tested architectures, it was selected as the final model for both the NumPy and PyTorch implementations.

The learning rate in all architecture experiments was fixed at the standard 0.01 and SGD was chosen to be used as the optimizer.

Table 1: Architecture Comparison (SGD, Learning Rate = 0.01)

Architecture	Validation Accuracy	Training Time (s)	Memory Usage (MB)
784-50-50-10	84.68%	46.13	266.75
784-64-64-10	84.39%	49.73	277.04
784-128-64-10	85.43%	68.02	301.53

3 Optimizer Experiments

After selecting the architecture, I experimented with different optimization techniques to further increase the accuracy.

The baseline implementation used standard Stochastic Gradient Descent (SGD). The convergence speed of SGD was low so I thought of using momentum as the optimizer to reduce convergence time

I then implemented the pure momentum optimization with a momentum coefficient of 0.9. Momentum helped reduce the convergence time significantly and increase the validation accuracy from 85.43% to 88.23%,

Seeing that pure momentum optimization performed well,I finally implemented Adam optimization o further reduce the convergence time as Adam combines Momentum and Rmsprop to improve optimization efficiency.

Using the standard Adam learning rate of 0.001 significantly reduced training time and achieved a validation accuracy of 87.11%. Although convergence was much faster, the accuracy was slightly lower than the Momentum implementation.

Seeing lr=0.01 perform better before in the previous architectures, I increased Adam's learning rate to 0.01. This produced the best overall balance between accuracy and convergence speed, achieving a validation accuracy of 89.02% while also having a low convergence time.

Seeing that modifying hyperparameters gave no fruitful results I chose this as my optimization technique.

Table 2: Optimizer Comparison for Architecture 784-128-64-10

Optimizer	Learning Rate	Validation Accuracy	Training Time (s)	Memory Usage (M
SGD	0.01	85.43%	68.02	301.53
Momentum (m = 0.9)	0.01	88.23%	36.31	301.95
Adam (lr = 0.001)	0.001	87.11%	2.69	302.36
Adam (lr = 0.01)	0.01	89.02%	3.91	302.36

4 Implementation Challenges

The most difficult part of the assignment was extending the neural network implementation to support a variable number of layers. Changing the forward pass implementation was relatively easy,but the backward pass introduced several indexing challenges and matrix mismatch.

Since the output layer used Softmax activation while all hidden layers used ReLU activation, I initially made mistakes while moving through the layers during backpropagation. Incorrect indexing caused matrix dimension mismatches and prevented gradients from propagating correctly through the network.

To debug this issue, I printed intermediate tensor dimensions and carefully checked the

computations being performed during backpropagation. This process helped me identify the incorrect indexing logic and correct my code. Debugging this problem helped me improve my understanding of how gradients flow through the network structure and helped me create a backpropagation implementation for a variable number of layers and neurons.

5 Performance Metrics

The performance of both implementations was evaluated using overall accuracy, loss curves, convergence curves, confusion matrices and class-wise accuracy on the Test Set and the results/plots are shown below.

5.1 Final Accuracy

Table 3: Final Test Set Comparison

Implementation	Test Accuracy	Training Time (s)	Memory Usage (MB)
NumPy (Scratch)	88.20%	123.97	356.96
PyTorch	89.14%	3.49	302.36

5.2 Training and Validation Loss Curves

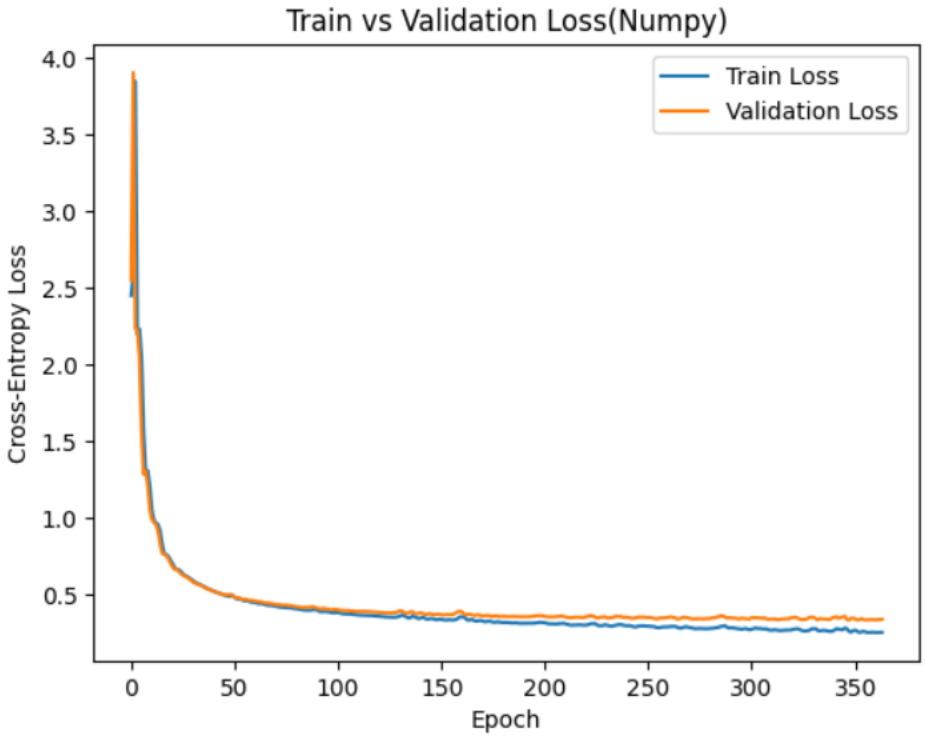

Figure 1: Training and Validation Loss Curves for the NumPy Implementation

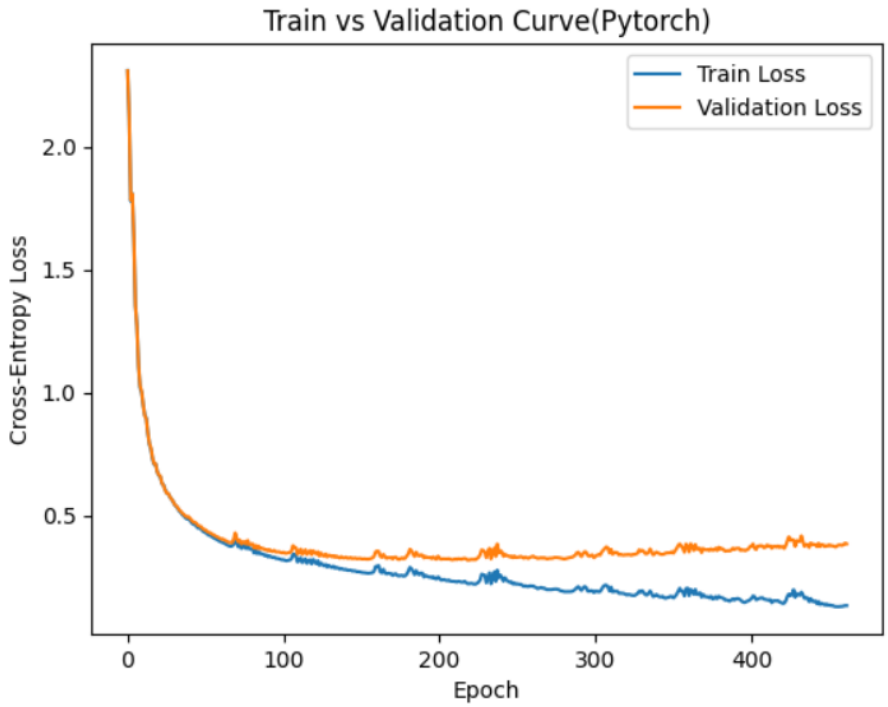

Figure 2: Training and Validation Loss Curves for the PyTorch Implementation

5.3 Convergence Curves

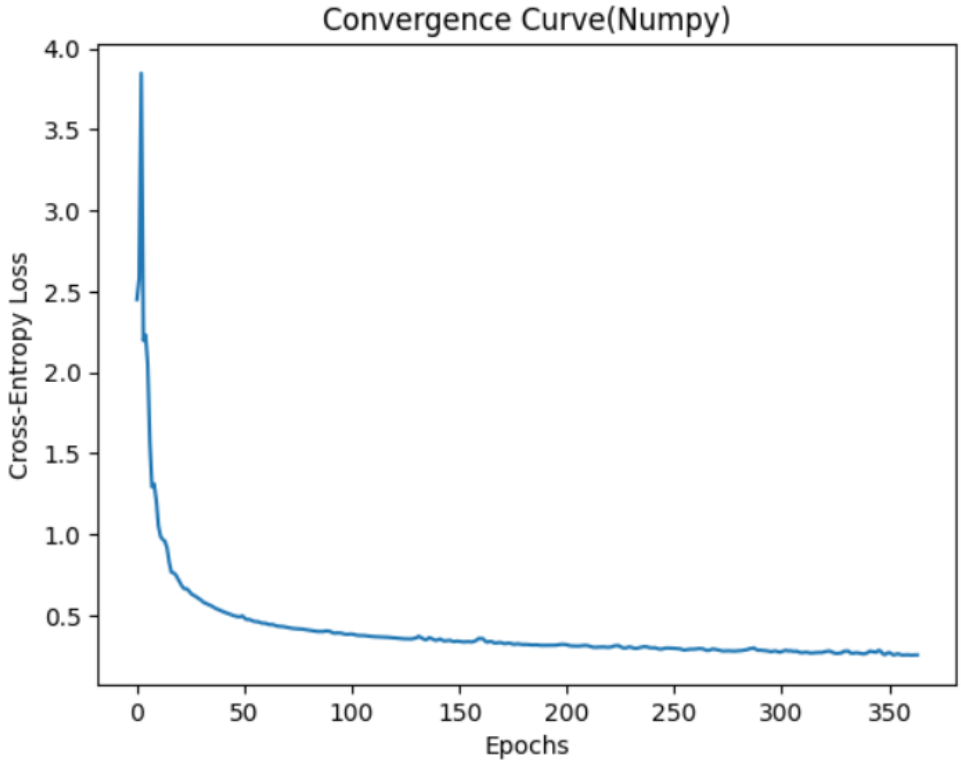

Figure 3: Convergence Behaviour of the NumPy Implementation

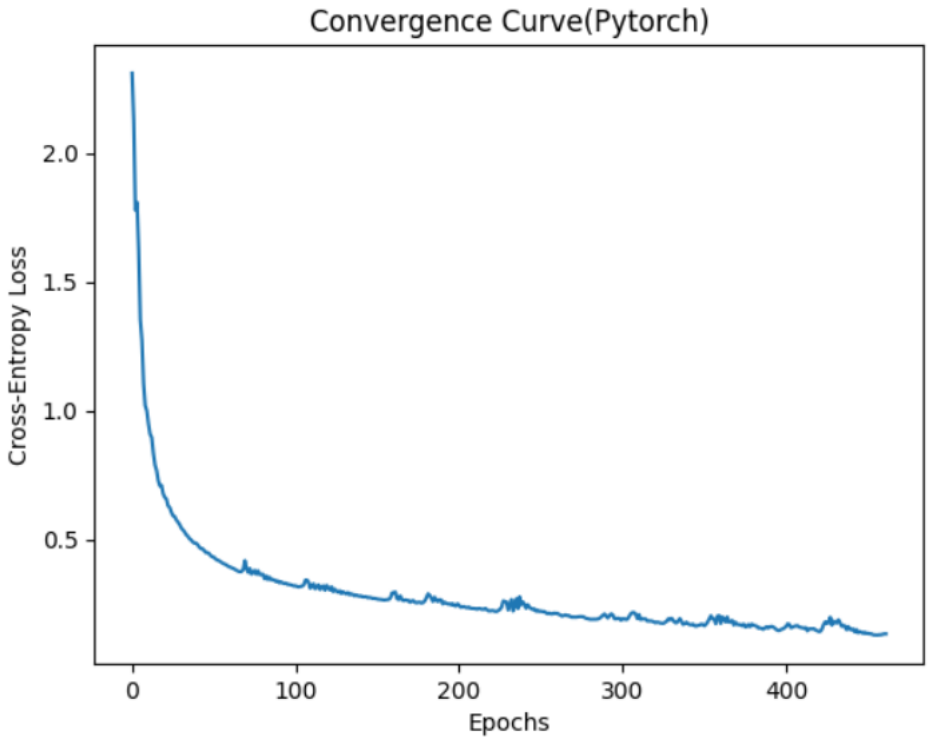

Figure 4: Convergence Behaviour of the PyTorch Implementation

5.4 Confusion Matrices

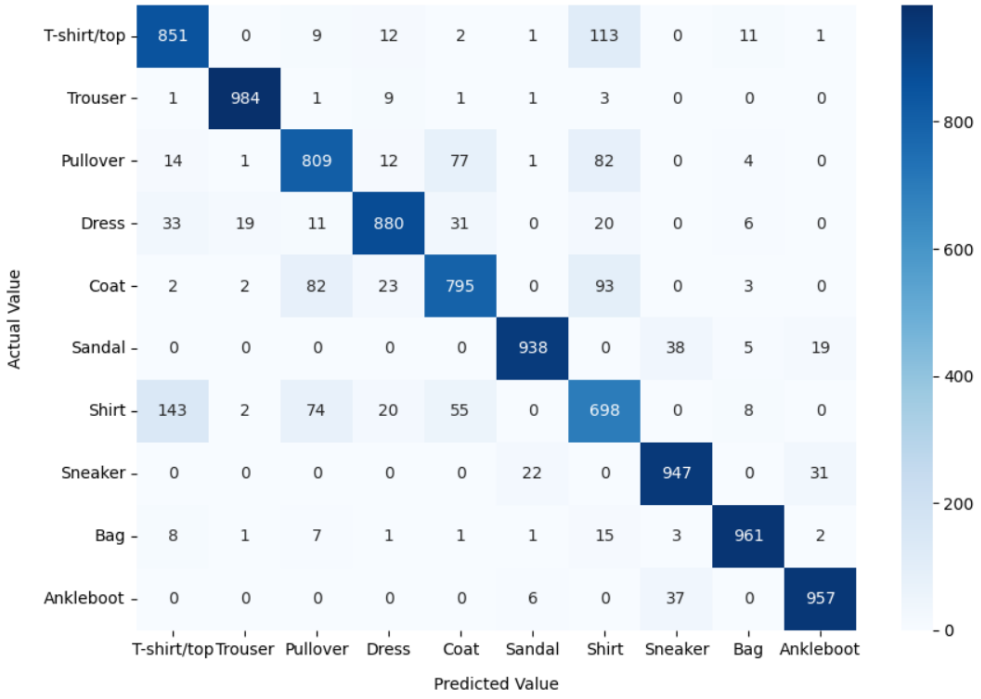

Figure 5: Confusion Matrix for the NumPy Implementation

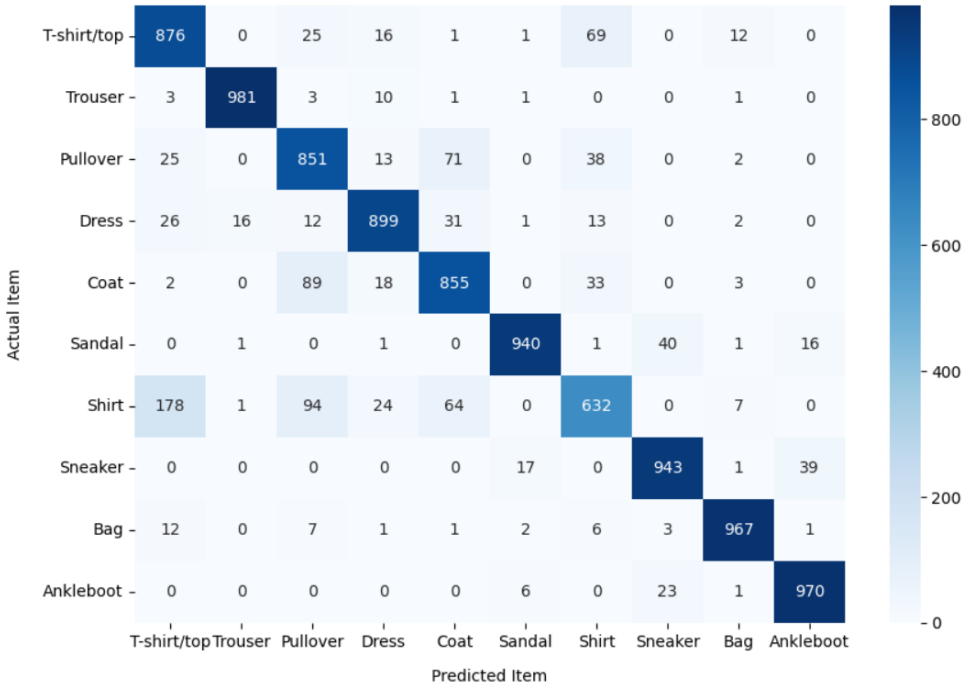

Figure 6: Confusion Matrix for the PyTorch Implementation

5.5 Class-wise Accuracy

Class-wise accuracy helps us determine the objects on which the model performs well and the objects on which it performs poorly.It along with the confusion matrix helps us realize when the model misclassifies the objects more.

Table 4: Class-wise Accuracy for the NumPy Implementation

Class	Accuracy (%)
T-shirt/top	85.1
Trouser	98.4
Pullover	80.9
Dress	88.0
Coat	79.5
Sandal	93.8
Shirt	69.8
Sneaker	94.7
Bag	96.1
Ankle Boot	95.7

Table 5: Class-wise Accuracy for the PyTorch Implementation

Class	Accuracy (%)
T-shirt/top	87.6
Trouser	98.1
Pullover	85.1
Dress	89.9
Coat	85.5
Sandal	94.0
Shirt	63.2
Sneaker	94.3
Bag	96.7
Ankle Boot	97.0

6 Analysis and Discussion

6.1 NumPy vs PyTorch Comparison

The difference in accuracy was very small as both the models use the same mathematical background, this also helps us validate the correctness of our neural network designed from scratch.The main difference was in the convergence speed of the two models.

The NumPy implementation got a test accuracy of 88.20% while taking approximately 124 seconds of training time.On the other hand, the PyTorch implementation achieved a slightly higher accuracy of 89.14% while requiring only 3.49 seconds of training time.This difference is due to PyTorch's highly optimized tensor operations. PyTorch performs matrix computations using optimized backend libraries and can efficiently use hardware(GPU) acceleration(when a GPU is available) to decrease its convergence time.

The PyTorch implementation was about 35 times faster than the NumPy implementation while also consuming less memory during training.

6.2 Learning Rate Effects

The learning also had an impact in the optimization of this neural network.

Using Adam with a learning rate of 0.001 resulted in a validation accuracy of 87.11%. Even though the convergence time reduced by a lot the validation accuracy didn't even reach that of the pure momentum implementation.

After increasing the learning rate to 0.01, the validation accuracy improved to 89.02% while also having a low convergence time.

This showed me that selecting an appropriate learning rate is important for having fast convergence and high accuracy.

6.3 Overfitting Analysis

In the PyTorch implementation, both training and validation losses decreased rapidly during the initial stages of training,and hence no overfitting.In the later epochs, the training loss decreased and validation loss slightly increased, which might have been due to mild overfitting. However, the model still showed a good test performance.

Similar things were seen for the NumPy implementation. The training and validation losses decreased throughout training, with a very small gap appearing between them in later epochs. This indicates very mild overfitting as the divergence was limited and the model performed good on the test set.

6.4 Gradient Analysis

I learned about the vanishing and exploding gradients problem which prevents the model from generalizing well and so to reduce the chances of this happening ReLU activation was used in all hidden layers. As it helps in stable learning through proper propagation of gradients.

The steady decrease in both training and validation loss showed me that gradients propagated properly throughout the network and no major gradient issue was seen.

6.5 Confusion Matrix Analysis

The confusion matrices helps us to understand that which items were easier and more difficult for the network to identify.Both implementations got a very high accuracy on classes such as Trouser, Bag, Sneaker and Ankle Boot, with Trouser getting over 98% accuracy. These item categories have distinct features, making them easier to identify.

The most low performing class was Shirt, achieving accuracies of 69.8% in the NumPy implementation and 63.2% in the PyTorch implementation. Items like Shirt, Pullover, Coat and T-shirt/top often have similar shapes,structure and features that is why they might be hard for the network to classify.

The confusion matrices were very similar in both implementations, showing that these errors arise from the limitations of ANNs as the networks weren't able to classify similar items easily.

7 Conclusion

From this task, I got a good experience on implementing a neural network from scratch. Architecture selection, experimenting with optimizers, and debugging the back propagation algorithm showed me how a neural network learns and what effect various changes can have on the network.In all the experiments, the choice of optimizer had a huge impact on the training process. The best accuracy and fastest convergence were achieved with Adam using a learning rate of 0.01. We got a slight variance in the Pytorch version than the Numpy one with the Pytorch version performing only a little better in terms

of accuracy,but the PyTorch version illustrated the benefits of optimized frameworks by having nearly identical results but converging much faster.